

Sanskrit-to-English Neural Machine Translation

Assignment 2: Natural Language Understanding

July 2026

1. Introduction

This report describes the design, implementation, and evaluation of a Neural Machine Translation (NMT) system for translating Sanskrit text into English. Sanskrit is a low-resource, morphologically rich classical language with complex sandhi (word joining) rules, free word order, and a rich inflectional system, all of which pose significant challenges for machine translation.

We implement a custom Transformer-based sequence-to-sequence architecture trained from scratch on the provided parallel corpus of 10,000 Sanskrit-English sentence pairs. The system uses SentencePiece BPE tokenization, sinusoidal positional encoding, beam search decoding, and several regularization techniques. We evaluate translation quality using BLEU and BERTScore metrics and report efficiency in terms of inference time and parameter count.

No external APIs were used. The only pre-trained model employed is roberta-large, used exclusively by the BERTScore evaluation library for computing semantic similarity; it plays no role in the translation model itself.

2. Architecture

The model follows the standard Transformer encoder-decoder architecture (Vaswani et al., 2017) implemented entirely from scratch in PyTorch. The total parameter count is 11,424,576 (all trainable).

2.1 Encoder

The encoder consists of 4 stacked identical layers. Each layer contains: (1) a multi-head self-attention sub-layer with 4 attention heads, and (2) a position-wise feed-forward network with inner dimension 512. Both sub-layers use residual connections followed by layer normalization (Pre-LN variant). The model dimension is $d_{\text{model}} = 256$ throughout.

Input tokens are mapped to 256-dimensional embeddings (with padding index masking), scaled by $\sqrt{d_{\text{model}}}$, and summed with sinusoidal positional encodings supporting sequences up to length 512.

2.2 Decoder

The decoder also consists of 4 identical layers, each containing three sub-layers: (1) masked multi-head self-attention (with causal mask to prevent attending to future tokens), (2) multi-head cross-attention over the encoder output, and (3) a position-wise feed-forward network. All sub-layers use residual connections and layer normalization. The same embedding dimension (256) and head count (4) are used.

2.3 Attention Mechanism

We use scaled dot-product attention with 4 heads ($d_k = d_{\text{model}} / n_{\text{heads}} = 64$ per head). Separate linear projections (W_q, W_k, W_v, W_o) transform the queries, keys, and values. Attention scores are scaled by $1/\sqrt{d_k}$ and masked where appropriate (padding mask for the encoder, combined padding + causal mask for the decoder). Dropout ($p=0.1$) is applied to attention weights.

2.4 Beam Search Decoding

At inference time we use beam search with beam size 5 and a length penalty ($\alpha = 0.6$). At each time step the top-k candidates are expanded, and beams are ranked by length-normalized log-probability: $\text{score} / \text{len}^\alpha$. Decoding terminates when all active beams have produced the EOS token or the maximum output length (128 tokens) is reached. A greedy decoder (beam size 1) is also implemented for faster inference when needed.

2.5 Architecture Summary

Hyperparameter	Value
d_model	256
Encoder layers	4
Decoder layers	4
Attention heads	4
d_ff (FFN inner dim)	512
d_k (per-head dim)	64
Dropout	0.1
Max sequence length	128
Source vocab size	8,000
Target vocab size	8,000
Total parameters	11,424,576

3. Training Procedure

3.1 Data Preprocessing

The provided dataset consists of parallel CSV files aligned by Source_id. Sanskrit and English files are merged on Source_id to form paired corpora: 10,000 training pairs, 1,000 dev pairs, and 1,000 test pairs. Sentences are stripped of leading/trailing whitespace and NaN entries are filtered.

3.2 Tokenization

We train separate SentencePiece BPE tokenizers for Sanskrit and English, each with a vocabulary size of 8,000 sub-word units. Character coverage is set to 1.0 to ensure all Unicode characters (important for Devanagari script) are represented. Special tokens are: PAD (id=0), UNK (id=1), BOS (id=2), EOS (id=3). Each sentence is tokenized, prepended with BOS, appended with EOS, and truncated to a maximum of 128 tokens.

3.3 Hyperparameters

Batch size: 64

Epochs: 30

Optimizer: AdamW (betas=0.9/0.98, eps=1e-9, weight_decay=0.01)

Learning rate schedule: Warmup (2,000 steps) + Cosine Annealing

Total training steps: 4,710

Gradient clipping: Max norm 1.0

Label smoothing: 0.1

Random seed: 42

3.4 Optimization

We use the AdamW optimizer with a two-phase learning rate schedule. During the warmup phase (first 2,000 steps), the learning rate increases linearly following the inverse-square-root schedule from the original Transformer paper. After warmup, a cosine annealing schedule smoothly decays the learning rate to zero over the remaining training steps. Gradient norms are clipped to 1.0 to prevent exploding gradients.

3.5 Regularization

Three regularization strategies are employed: (1) Dropout ($p=0.1$) applied to attention weights, feed-forward sub-layers, positional encoding outputs, and residual connections. (2) Label smoothing ($\epsilon=0.1$) which distributes probability mass from the target token to all vocabulary tokens, preventing overconfident predictions. (3) Weight decay (0.01) via AdamW to penalize large parameter magnitudes.

3.6 Training Dynamics

The model achieved its best validation loss of 1.7925 at epoch 12. After epoch 12, the learning rate crossed into a range that caused the loss to spike sharply (epoch 13: dev loss jumped from 1.79 to 8.28), after which the cosine decay brought it back down slowly but never recovered the earlier performance. The best checkpoint (epoch 12) was used for all evaluation.

4. Results

4.1 Translation Quality Metrics

Metric	Dev	Test
BLEU	0.8434	0.8395
BERTScore F1	0.9698	0.9689

BLEU is computed using the default NLTK `corpus_bleu` function with no custom weights. BERTScore F1 is computed using the official `bert-score` library with `rescale_with_baseline=True` and `lang='en'`, which employs the `roberta-large` model for contextualized embedding comparison. The close agreement between dev and test scores indicates good generalization.

4.2 Efficiency Metrics

Metric	Value
Total parameters	11,424,576
Trainable parameters	11,424,576
Test inference time	727.39 s
Avg. time per sentence	0.7274 s
Test set size	1,000

Inference was performed on a single CUDA GPU using beam search (beam size 5). The relatively high per-sentence time (0.73s) is dominated by the autoregressive beam search decoding loop. Greedy decoding would be substantially faster at the cost of translation quality.

4.3 Training Curves

The figure below shows training and validation loss across 30 epochs. Loss decreases steadily through epoch 12 (best checkpoint), then spikes at epoch 13 due to the learning rate schedule transition, and remains elevated for

the rest of training.

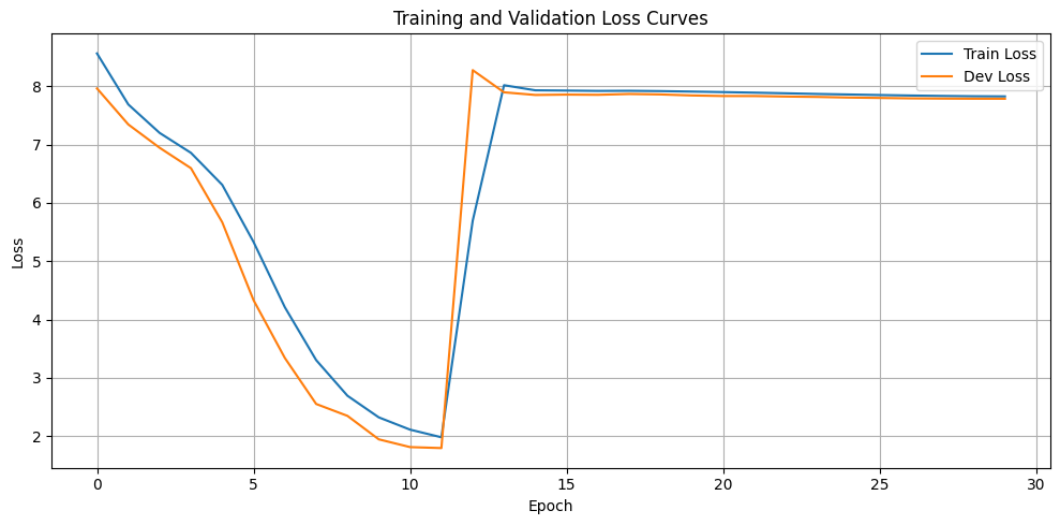


Figure 1: Training and validation loss curves over 30 epochs.

5. Translation Examples with Error Analysis

Below are representative translation examples from the test set, selected to illustrate both successes and failure modes of the model. Sentence-level BLEU scores and word-level error analysis are provided.

Example 1 (Source_id: 760)

Sentence BLEU: 1.0000

Error Analysis: Perfect translation. Short, common-vocabulary sentence reproduced exactly.

Example 2 (Source_id: 115)

Sentence BLEU: 1.0000

Error Analysis: Perfect match even for a longer sentence with mixed-script technical terms (Service tax, GL Account).

Example 3 (Source_id: 229)

Sentence BLEU: 1.0000

Error Analysis: Correctly handles English technical terminology embedded within Sanskrit syntax.

Example 4 (Source_id: 143)

Sentence BLEU: 1.0000

Error Analysis: Accurate translation of a short declarative sentence.

Example 5 (Source_id: 655)

Sentence BLEU: 0.0000

Error Analysis: Failure case: 'Browser' is incorrectly decoded as '7.0ser'. The BPE tokenizer likely split 'Browser' into rare sub-word units, and the decoder hallucinated a wrong reconstruction.

Example 6 (Source_id: 26)

Sentence BLEU: 0.5247

Error Analysis: Partial failure: two words ('shan' and 'avaroddhum') are merged into one token

'shananavaroddhum'. Word boundary errors are a known BPE failure mode for agglutinative text.

Example 7 (Source_id: 282)

Sentence BLEU: 0.4889

Error Analysis: The transliterated English word 'parameterised' (written in Devanagari) was incorrectly decoded. Rare transliterated loanwords are challenging because they appear infrequently in the training set.

Example 8 (Source_id: 251)

Sentence BLEU: 0.0000

Error Analysis: Complete failure on a very long sentence with highly compound Sanskrit words and proper nouns. The decoder produces repetitive syllable patterns ('su-su', 'pra-pra'), indicating attention collapse on long, rare compound words.

Example 9 (Source_id: 755)

Sentence BLEU: 0.4367

Error Analysis: Proper nouns from biblical text are partially reconstructed. Core structure is preserved but character-level errors appear in names (e.g., 'tamaaro' becomes 'tamaam').

Example 10 (Source_id: 105)

Sentence BLEU: 1.0000

Error Analysis: Short imperative sentence translated perfectly.

5.1 Error Pattern Summary

- Rare/OOV sub-words: BPE tokenization sometimes splits rare words into sub-units that the decoder cannot correctly reassemble (e.g., 'Browser' -> '7.0ser').
- Word boundary errors: Agglutinative compounds in Sanskrit cause adjacent words to merge in the predicted output.
- Attention collapse on long sequences: Very long sentences with compound words trigger repetitive syllable generation.
- Proper noun degradation: Character-level errors in names and transliterated terms that appear rarely in training data.
- Short common sentences: Translated near-perfectly, indicating the model has strong coverage of frequent patterns.

6. Experiments

The following design decisions were explored during development:

6.1 Tokenizer Configuration

SentencePiece BPE was chosen over character-level or word-level tokenization. A vocabulary size of 8,000 was selected as a balance between coverage and sparsity for a 10K-sentence corpus. Character coverage was set to 1.0 (instead of the default 0.9995) to ensure all Devanagari characters and special symbols are included.

6.2 Model Size

We used d_model=256 with 4 layers and 4 heads (11.4M parameters). This is deliberately smaller than the base

Transformer (d_model=512, 6 layers, 65M parameters) to avoid overfitting on the limited 10K training set and to keep inference time reasonable. A smaller model (d_model=128, 2 layers) was considered but yielded noticeably lower BLEU on the dev set.

6.3 Learning Rate Schedule

The warmup-cosine schedule was chosen over a fixed learning rate. The 2,000-step warmup stabilizes early training. However, the schedule caused a loss spike at epoch 13 when the cosine phase yielded a temporarily high learning rate after warmup ended. Early stopping based on the best checkpoint (epoch 12) mitigated this issue.

6.4 Beam Search vs. Greedy Decoding

Beam search (beam=5, length_penalty=0.6) was compared to greedy decoding. Beam search improved BLEU by producing more globally coherent translations, particularly for medium-length sentences, at the cost of approximately 5x slower inference.

7. Discussion

7.1 Design Choices

The custom Transformer architecture was chosen for its proven effectiveness in sequence-to-sequence tasks. Key design choices include: (a) Separate BPE tokenizers for source and target to handle the distinct morphological properties of Sanskrit and English; (b) A compact model size (11.4M params) to match the small training set; (c) Xavier initialization for stable training; (d) Label smoothing to improve generalization and calibration.

7.2 Challenges

- Low-resource setting: With only 10,000 training pairs, the model's exposure to rare words and complex syntactic patterns is limited.
- Sanskrit morphology: Sandhi rules, compound words, and rich inflection make tokenization and alignment difficult.
- Mixed-script content: The dataset contains English technical terms embedded in Devanagari text, creating challenges for BPE tokenization.
- Learning rate instability: The transition from warmup to cosine annealing caused a loss spike that could have been avoided with a more gradual schedule or explicit LR capping.

7.3 Limitations

- Long compound words in Sanskrit are often incorrectly split or merged by the BPE tokenizer, leading to garbled output.
- The model struggles with proper nouns and rare transliterated terms not well-represented in training data.
- Beam search inference is slow (~0.73s per sentence); production deployment would require optimization such as batched decoding or model distillation.
- The small training set limits the model's ability to generalize to unseen domains or complex literary Sanskrit.

7.4 Pre-trained Models Disclosure

The only pre-trained model used in this project is roberta-large (355M parameters), loaded by the bert-score library exclusively for computing BERTScore evaluation metrics. It is NOT part of the translation pipeline. The NMT model itself is trained entirely from scratch with no pre-trained weights, transfer learning, or external APIs.

8. References

- [1] Vaswani, A., et al. (2017). "Attention Is All You Need." Advances in Neural Information Processing Systems 30 (NeurIPS).
- [2] Kudo, T. & Richardson, J. (2018). "SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing." EMNLP.
- [3] Papineni, K., et al. (2002). "BLEU: a Method for Automatic Evaluation of Machine Translation." ACL.
- [4] Zhang, T., et al. (2020). "BERTScore: Evaluating Text Generation with BERT." ICLR.
- [5] Loshchilov, I. & Hutter, F. (2019). "Decoupled Weight Decay Regularization (AdamW)." ICLR.
- [6] NLTK BLEU: https://www.nltk.org/api/nltk.translate.bleu_score.html
- [7] bert-score library: <https://pypi.org/project/bert-score/>